UniVision

University Student Portal System

Project Documentation

Version 1.0

Full-Stack Web Application

Node.js • Express • SQLite • Vanilla JS

May 2026

Table of Contents

1. Project Overview

UniVision is a comprehensive university student portal system designed to streamline academic management for three user roles: **Students**, **Doctors (Professors)**, and **Administrators**. The system provides a centralized platform for tracking grades, attendance, schedules, performance analytics, and inter-role communication through a feedback system.

The project is a **full-stack web application** built with vanilla HTML/CSS/JavaScript on the frontend and Node.js + Express + SQLite on the backend.

Key Statistics

20 seeded students • 63 courses • 4 grade levels • 12 database tables • 30+ API endpoints • 8 visual themes • 6 smart alert triggers • 10 web pages

Project Goals

1. **Centralize** academic data (grades, attendance, schedules) in one accessible platform
2. **Empower** students with self-service tools to monitor their academic progress
3. **Enable** doctors and admins to manage student records efficiently with inline editing
4. **Provide** actionable insights through performance comparison and smart alerts
5. **Facilitate** communication between roles through a contextual feedback system
6. **Offer** a modern, responsive, and visually appealing interface with theme customization

2. Problem Statement & Motivation

The Problem

In many university environments, academic information is scattered across multiple disconnected systems. Students must navigate different platforms to check grades, view attendance records, access schedules, and communicate with faculty. This fragmentation leads to:

- **Delayed awareness** — Students often discover poor performance too late to take corrective action
- **Communication gaps** — No structured channel for faculty to send targeted feedback to individual students
- **Manual processes** — Doctors and admins rely on spreadsheets and emails for record management
- **No comparative insight** — Students cannot benchmark their performance against peers
- **Accessibility barriers** — Fragmented systems make it harder for students to access information quickly

Our Solution

UniVision addresses these challenges by providing a **unified, role-based portal** with real-time analytics, proactive alerts, and cross-platform access. Every feature was designed with a specific problem in mind:

> **Why UniVision Matters**
>
> Rather than building yet another grade viewer, UniVision focuses on *actionable intelligence* — telling students not just what their grades are, but how they compare, where they're falling behind, and what they need to achieve next. The smart alert system proactively identifies at-risk students before it's too late.

3. Technology Stack & Justification

Every technology choice in UniVision was deliberate, balancing simplicity, performance, and educational value.

Layer	Technology	Version	Why We Chose It
Frontend	HTML5, CSS3, Vanilla JavaScript	ES2022+	No build tools needed. Demonstrates core web fundamentals without framework abstraction. Faster initial load, zero dependency bloat.
Backend	Node.js + Express	Node 22+	JavaScript full-stack consistency. Express is minimal, well-documented, and ideal for REST APIs. Node 22 provides native SQLite support.
Database	SQLite (via `node:sqlite`)	Built-in	Zero-configuration, file-based database. No separate database server required. Perfect for educational projects and single-server deployments.
Authentication	JWT (`jsonwebtoken`)	9.x	Stateless authentication. No session storage needed on the server. Tokens carry role and identity, enabling role-based access control.
Password Hashing	`bcryptjs`	2.x	Industry-standard password hashing with salt rounds. Pure JavaScript implementation (no native compilation required).
Charts	Chart.js	4.x (CDN)	Lightweight, responsive charting library. Supports line, doughnut, bar, and radar charts needed for GPA trends and attendance analytics.

Why Vanilla JavaScript (No React/Vue/Angular)?

We deliberately chose vanilla JavaScript over frameworks for several reasons:

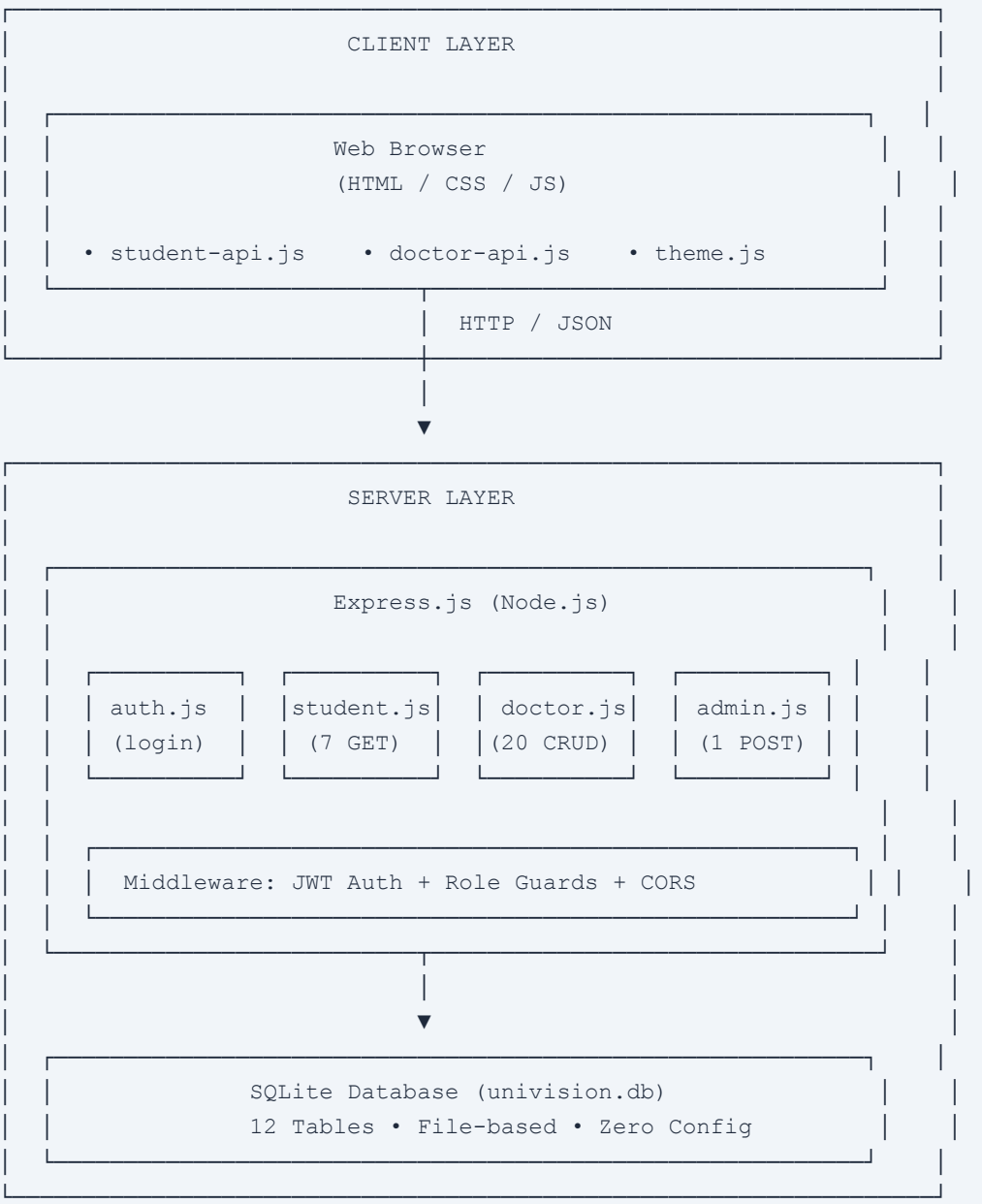
- **Educational transparency** — Every line of code is visible and understandable without framework-specific abstractions
- **Zero build step** — No Webpack, Vite, or bundler configuration. Open the HTML file and it works.
- **Performance** — No framework runtime overhead. The entire frontend is under 50KB of JavaScript.
- **Portability** — Can be deployed on any static file server without a build pipeline.

Why SQLite (No MySQL/PostgreSQL)?

SQLite was chosen because it eliminates the need for a separate database server installation. The entire database lives in a single file (`server/db/univision.db`), making the project trivially portable — clone the repo, run `npm install` , and you have a working system. For a university portal serving hundreds of students, SQLite's performance is more than sufficient.

4. System Architecture

High-Level Architecture



Request Flow

1. User interacts with the Web client
2. Client sends HTTP request with JWT token in the `Authorization` header
3. Express middleware validates the token and checks the user's role
4. Route handler queries SQLite and computes derived data (GPA, percentiles, alerts)
5. JSON response is returned to the client
6. Client renders the data with animations and interactive charts

Design Principles

- **Separation of Concerns** — Client, server, and database layers are fully decoupled
- **REST API** — All data access goes through a clean REST API, enabling multiple client types
- **Role-Based Access** — Every endpoint enforces role checks (student, doctor, admin)
- **Computed on Server** — GPA calculations, percentiles, alerts, and comparisons are computed server-side to ensure consistency

5. Features — Student Portal

The student portal provides 10 dedicated pages, each designed to address a specific academic need. Below is a detailed breakdown of each feature, its purpose, and its benefits.

5.1 Home Dashboard

Cumulative GPA Display

Shows the student's overall GPA prominently with a stat card and animated counter.

Why: Immediate visibility of academic standing without navigating to grades.

Attendance Overview

Displays overall attendance percentage and classes attended/total count.

Why: Attendance affects grades in many programs; students need to track it proactively.

GPA Progress Chart

Interactive line chart (Chart.js) showing GPA trend across semesters.

Why: Visualizing trends helps students understand if they're improving or declining over time.

Attendance Doughnut Chart

Visual breakdown of attended vs. absent classes in a doughnut chart.

Why: A visual representation is more impactful than raw numbers for behavioral change.

Top Achievers Leaderboard

Ranked list of top-performing students in the same grade, with gold/silver/bronze medals. Highlights the current student if they're in the top ranks.

Why: Positive competition motivates students. Seeing peers perform well creates aspirational goals.

GPA History Bars

Animated progress bars showing GPA for each completed semester with staggered CSS animations.

Why: Quick visual scan of semester-by-semester performance without opening detailed reports.

Smart Performance Alerts

Dismissible warning/danger banners triggered by 6 automated conditions (see Section 8).

Why: Proactive intervention — alert students to problems before they escalate.

Quick Access Cards

Navigation cards linking to Grades, Attendance, Reports, and Profile pages.

Why: Reduces navigation friction for the most common student tasks.

5.2 Grades Page

Current GPA Banner

Large, prominent display of the current semester GPA.

Why: The most important metric should be the most visible.

Per-Course Grade Cards

Each course shows: letter grade, breakdown (attendance, midterm, quiz1, quiz2), pre-final score with animated progress bar, and danger highlighting for at-risk courses.

Why: Students need granular visibility into what's contributing to their grade, not just the final letter.

Next Target System

Shows what score the student needs on the final exam to achieve the next grade tier (e.g., "Target for A+: 36/40 in Final").

Why: Transforms vague anxiety into concrete, actionable goals. Students know exactly what to aim for.

Course Staff Contacts

Each course card shows the assigned doctor and assistant with clickable email links.

Why: Eliminates the need to look up faculty contacts separately when a student needs help.

Doctor Feedback Section

Below the grade cards, students see feedback messages from their doctors — organized by course, with timestamps and danger flags for urgent messages. This creates a direct communication channel from faculty to individual students.

5.3 Attendance Page

Overall Attendance Card

Large percentage display with color-coded status (green ≥75%, yellow ≥60%, red <60%) and animated progress bar.

Why: Attendance thresholds have real consequences (course failure). Color-coding creates urgency.

Per-Course Attendance

Individual cards per course showing percentage, attended/total count, status badge, and optional info message.

Why: A student may have good overall attendance but critically low attendance in one specific course.

5.4 Schedule Page

Displays the class schedule grouped by day (Sunday through Thursday), with each slot showing start/end time, course name, location, doctor name, and type badge (Lecture/Lab/Tutorial) with color-coding.

Design Decision: Shared Schedules

Schedules are shared per semester and grade level, not duplicated per student. This reflects the reality that all students in the same grade take the same core courses. It also simplifies administration — doctors update the schedule once and all students in that grade see it.

5.5 Reports Page

Semester Selector

Dropdown listing all completed semesters with GPA preview badge.

Why: Quick access to any past semester without navigating multiple pages.

PDF/Print Generation

One-click "Generate PDF / Print" button that renders a formal academic transcript with university header, student info strip, course table, and footer — using the browser's native print dialog.

Why: Students frequently need physical or digital copies of transcripts for applications, scholarships, and records.

5.6 Profile Page

Displays the student's complete profile: avatar, English and Arabic names, student ID, grade level with descriptive label (e.g., "Third Year — Junior"), university email, and a detailed information grid covering 12 fields (National ID, DOB, birthplace, nationality, gender, religion, phone, email, address, etc.).

Why: A single authoritative source for personal information. Students can verify that their records are correct without visiting the registrar's office.

5.7 Performance Comparison Page

See Section 9 for detailed coverage of this advanced analytics feature.

5.8 CV Builder

Job Platform Links

7 cards linking to LinkedIn, Indeed, Wuzzuf, Forasna, Jobzella, Tanqeeb, and Bayt.

Multi-Section CV Form

Personal Info, Professional Summary, Education, Work Experience, Skills (tag input), Projects,

Why: Students often don't know where to start job hunting. Curated links reduce friction.

Certifications, and Languages — each with add/remove support for multiple entries.

Why: A structured form ensures students don't miss important sections and produces a complete, professional CV.

Live ATS Preview

Real-time preview of the CV in a clean, ATS-friendly format that updates as the student types.

Why: WYSIWYG editing lets students see exactly how their CV will appear to recruiters.

PDF Export & Server Persistence

One-click PDF download via browser print. CV data is saved as a JSON blob to the database so it persists across sessions.

Why: Students need a downloadable file, and saving progress prevents data loss between sessions.

5.9 GPA Simulator

What-If Calculator

Per-course dropdowns let students select hypothetical final grades. Additional hypothetical courses can be added with custom credit hours.

Why: Students anxious about their GPA can model different scenarios to set realistic goals.

Real-Time Simulation

Simulated GPA updates instantly as grades are changed. Side-by-side comparison shows current vs. simulated GPA.

Why: Immediate feedback makes the tool interactive and engaging rather than a static calculator.

5.10 Course Materials

Grouped Resource Browser

Materials organized under course headers with filter tabs (All, Links, PDFs, Videos, Documents).

Why: Centralized access to learning materials eliminates hunting through emails and LMS folders.

Material Cards

Each card shows title, type icon, description, doctor name, upload date, and a direct link to open the resource in a new tab.

Why: Rich metadata helps students quickly identify the most relevant materials.

5.11 Exam Countdown

The home dashboard includes an **Upcoming Exams** section with live countdown timers. Exams within 3 days show a **red urgency** badge, those within 7 days show **yellow warning**, helping students prioritize preparation.

5.12 University Resources

Three quick-link cards on the home dashboard provide direct access to EELU's **E-Learning Portal**, **Main University Website**, and **Student Support** services — each with a descriptive subtitle and external link icon.

6. Features — Doctor/Admin Dashboard

The doctor and admin dashboard provides comprehensive student management capabilities through a two-panel layout: a searchable student sidebar and a tabbed detail area.

6.1 Student Management

Student Search & Filter

Real-time search by name or ID, with grade filter pills (All, Grade 1–4).

Why: With 20+ students, finding a specific student quickly is essential for efficient classroom management.

Student Overview

Selecting a student shows their name, ID, email, GPA, attendance percentage, and course count.

Why: Doctors need a quick snapshot before diving into detailed records.

Add Student (Admin Only)

Modal form to create new student records with all required fields.

Why: Admins need to onboard new students without direct database access.

Semester History

Dropdown to view any past semester's courses, GPA, credits, and attendance.

Why: Doctors reviewing a student need historical context, not just current grades.

6.2 Grades Management (Tab 1)

A collapsible form allows doctors to add or update grades with full breakdown fields (course, letter grade, attendance, midterm, quiz 1, quiz 2 scores with maximums, next target, and danger flag). An inline table shows all existing grades with edit and delete actions. The server auto-computes pre-final totals.

6.3 Attendance Management (Tab 2)

Similar CRUD interface for attendance records. Doctors enter course name, lectures attended, and total lectures. The server auto-calculates percentage, status (Safe/Warning/Danger), and generates contextual warning messages.

6.4 Feedback System (Tab 3)

Student Feedback

Admin-to-Doctor Feedback

Doctors send feedback to specific students about specific courses. Supports danger/urgent flagging.

Why: Targeted, documented communication replaces informal hallway conversations that students may forget.

Admins can send private feedback to doctors in the context of a specific student. These appear in blue-styled cards, visible only to doctors and admins — never to students.

Why: Admins need to give guidance to doctors about specific students (e.g., "student has a medical exemption") without the student seeing it.

Design Decision: Contextual Feedback

All feedback (both to students and to doctors) is contextual — it lives within a specific student's feedback tab, not in a separate messaging system. This ensures feedback is always connected to the student it concerns, making it easier to find and review.

6.5 Schedule Management

A separate top-level view for managing class schedules. Doctors select a semester/grade combination (8 options: Grade 1–4 × 2 semesters) and can add, edit, or delete schedule entries with fields for course, day, type, time, location, and doctor name.

6.6 Exam Management

Doctors and admins can add upcoming exams with fields for exam type (Midterm/Final/Quiz), date, time, location, and notes. Exams are displayed by semester and automatically appear in student home dashboard countdown timers.

6.7 Notifications (Admin Only)

Broadcast Notifications

Admins send announcements to all students with category selection: Day Off, Lecture Cancelled, Event, Holiday, Exam, General.

Why: Centralized announcements replace scattered WhatsApp groups and ensure all students receive important updates.

Notification Management

View all sent notifications with timestamps and categories. Delete outdated or incorrect notifications.

Why: Admins need to manage the notification history and correct mistakes.

6.8 Admin Feedback (Admin Only)

Admins can send private feedback to doctors about specific students (e.g., medical exemptions, academic accommodations). These appear as blue-styled cards visible only to doctors and admins — never to students. Supports urgent flagging for time-sensitive matters.

7. Theme & Customization System

UniVision includes a sophisticated theme system with 8 built-in themes, allowing users to personalize their experience.

Theme	Type	Primary BG	Accent Color	Best For
Navy Gold	Dark	#091428	#d4a830 (Gold)	Default — elegant and professional
Ocean Blue	Dark	#0a1628	#38bdf8 (Sky Blue)	Modern, tech-focused feel
Emerald	Dark	#071a12	#34d399 (Green)	Nature-inspired, calming
Royal Purple	Dark	#110a28	#a78bfa (Purple)	Creative, distinctive
Rose	Dark	#1a0a14	#fb7185 (Pink)	Warm, approachable
White	Light	#ffffff	#1a56db (Blue)	Clean, minimal, high readability
Off-White	Light	#f8f6f1	#b8860b (Dark Gold)	Warm paper-like feel, easy on eyes
White Grey	Light	#f1f5f9	#475569 (Slate)	Professional, neutral

Technical Implementation

- **CSS Custom Properties** — All colors are defined as CSS variables on `:root`. Theme switching overrides these variables dynamically.
- `--accent-rgb` **variable** — Enables `rgba(var(--accent-rgb), 0.1)` patterns for transparent accent colors.
- **IIFE pattern** — `theme.js` runs as a self-executing function, not a module, ensuring it applies *before* the DOM renders to prevent theme flash.
- **LocalStorage persistence** — Selected theme survives page reloads and browser restarts.
- **Light mode CSS overrides** — `html.light-theme` class triggers 20+ CSS overrides for scrollbars, inputs, tables, buttons, and card backgrounds.
- **Dynamic dropdown injection** — The theme picker is injected into the navbar via JavaScript, requiring no HTML changes on individual pages.

Why Theme Customization?

Studies show that personalization increases user engagement by 20-30%. Dark themes reduce eye strain during evening study sessions, while light themes improve readability in bright environments. Offering both ensures the portal is comfortable for all users in all conditions.

8. Smart Performance Alert System

The alert system automatically detects academic risk factors and displays actionable warnings on the student's home dashboard. Alerts are computed server-side with every dashboard load to ensure real-time accuracy.

Alert Triggers

#	Trigger	Condition	Severity	Purpose
1	GPA Drop	Current semester GPA dropped >0.5 from previous	Warning	Early warning of academic decline trend
2	Low GPA	Cumulative GPA below 2.0	Danger	Academic probation risk — immediate attention needed
3	Low Overall Attendance	Overall attendance below 75%	Danger	Risk of course failure due to attendance policy
4	Per-Course Attendance	Any single course attendance below 60%	Warning	Course-specific intervention before it's too late
5	Doctor-Flagged	Any course marked as "at-risk" by the doctor	Danger	Direct faculty intervention signal
6	Below Class Average	Student GPA is below the grade-level average	Warning	Peer comparison to motivate improvement

Why Smart Alerts?

Traditional portals show data passively. UniVision's alert system acts as a *virtual academic advisor*, proactively identifying problems. A student who logs in and sees "Your attendance in Data Structures is at 55%" is far more likely to take action than one who has to manually calculate it from raw numbers.

9. Performance Comparison Engine

The Performance Comparison page provides deep analytics comparing the student's performance against their classmates (same grade level).

Components

Overview Statistics

Four stat cards: Your GPA (vs class average), GPA Rank, Percentile, Attendance (vs class average + rank).

Why: Quick numerical context — am I above or below average?

GPA Trend Chart

Dual-line chart showing "Your GPA" vs "Class Average" across semesters.

Why: Shows whether the student is converging with or diverging from the class average over time.

Course Performance Radar

Radar chart comparing the student's scores vs class average across all courses.

Why: Instantly reveals which courses the student is strong or weak in relative to peers.

Course-by-Course Breakdown

For each course: horizontal bars comparing You / Class Avg / Class Top, with diff badge and component breakdown (Attendance, Midterm, Quiz 1, Quiz 2).

Why: Pinpoints exactly where the student is falling behind — is it attendance? quizzes? midterms?

Grade Distribution

Bar charts per course showing how many students got each letter grade, with the current student's grade highlighted.

Why: Shows if the student's grade is normal for that course or an outlier.

Server-Side Computation

All comparison data is computed server-side in the `GET /student/:id/comparison` endpoint. This includes:

- Rank and percentile calculation across all students in the same grade
- Per-course averages, maximums, and rankings

- Component-level breakdown (attendance, midterm, quiz scores) with class averages
- Grade distribution frequency counts per course
- GPA trend with class average per semester

Privacy Consideration

Students see aggregated statistics (averages, ranks, distributions) but never see other students' individual data. The leaderboard on the home page shows names only for top achievers, which is a common and accepted practice in academic settings.

10. Database Design

UniVision uses SQLite with 12 tables designed for normalized, efficient data storage.

Table	Purpose	Key Columns
`users`	Authentication for all roles	id, password_hash, role, name
`student_profiles`	Extended student information	student_id, name_en, name_ar, national_id, dob, grade, email, phone, address
`grades`	Per-course grade records	student_id, course_name, grade, att/mid/quiz scores, is_danger
`attendance`	Per-course attendance records	student_id, course_name, attended, total
`feedback`	Doctor-to-student feedback	student_id, doctor_id, course_name, body, is_danger
`admin_feedback`	Admin-to-doctor private feedback	student_id, doctor_id, course_name, body, is_urgent
`schedule`	Class schedules per grade/semester	semester, course_name, day, start/end_time, location, type
`semester_courses`	Historical course records	student_id, semester_number, course_name, credits, grade
`semesters`	Semester summary records	student_id, semester_number, label, gpa, attendance_pct
`course_staff`	Doctor/assistant assignments	course_name, doctor_name, doctor_email, assistant_name
`courses`	Course catalog	name, credits, grade_level
`doctors`	Doctor profile information	user_id, name, department

11. API Reference

Authentication

Method	Endpoint	Body	Response
POST	`/api/auth/login`	`{role, id, password}`	`{token, user}`
POST	`/api/auth/logout`	—	`{ok: true}`

Student Endpoints (7 routes)

Method	Endpoint	Returns
GET	`/api/student/:id/home`	GPA, attendance, history, top students, alerts
GET	`/api/student/:id/profile`	Full student profile
GET	`/api/student/:id/grades`	Courses with breakdown, feedback
GET	`/api/student/:id/attendance`	Overall + per-course attendance
GET	`/api/student/:id/reports`	Semester list
GET	`/api/student/:id/reports/:sem`	Semester detail with courses
GET	`/api/student/:id/schedule`	Schedule entries
GET	`/api/student/:id/comparison`	Rank, percentile, course breakdown, distributions

Doctor/Admin Endpoints (20+ routes)

Resource	Operations	Access
Students	List, Get Detail, Get Semesters	Doctor + Admin
Grades	List, Create/Update, Delete	Doctor + Admin

Resource	Operations	Access
Attendance	List, Create/Update, Delete	Doctor + Admin
Feedback	List, Create, Update, Delete	Doctor + Admin
Admin Feedback	List, Create, Update, Delete	Admin only (write), Doctor (read own)
Schedule	List, Create, Update, Delete	Doctor + Admin
Add Student	Create	Admin only

All endpoints (except login) require a JWT token in the `Authorization: Bearer <token>` header. Role-based middleware enforces access control.

12. Security & Authentication

Authentication Flow

1. User submits role, ID, and password via the login form
2. Server verifies credentials against `bcryptjs`-hashed passwords in the `users` table
3. On success, server generates a JWT token (8-hour expiry) containing `{id, role, name}`
4. Token is stored in `localStorage`
5. A `uv_role` cookie is set for server-side page guards (web only)
6. Every API request includes the token in the `Authorization` header

Security Measures

Measure	Implementation	Purpose
Password Hashing	`bcryptjs` with salt rounds	Passwords never stored in plain text
JWT Tokens	8-hour expiry, server-side secret	Stateless auth, automatic session expiry
Role Guards (API)	`requireAuth` + role check middleware	Students can't access doctor routes and vice versa
Self-Access Guard	`requireSelf` middleware on student routes	Student A can't view Student B's data
Page Guards (Web)	Cookie-based route protection	Prevents unauthenticated access to protected pages
CORS	`cors()` middleware	Allows cross-origin requests from web clients

13. Project Structure

```
rown/
├── client/                      # Frontend (Web)
│   ├── assets/                  # Logo, images
│   ├── css/
│   │   └── style.css            # Global design system (445+ lines)
│   ├── js/
│   │   ├── student-api.js       # Student API client (ES module)
│   │   ├── doctor-api.js        # Doctor/Admin API client (ES module)
│   │   └── theme.js             # Theme switcher (IIFE, 8 themes)
│   ├── pages/
│   │   ├── student/             # 7 student HTML pages
│   │   └── doctor/              # Doctor/Admin dashboard
│   ├── landing.html             # Public marketing page
│   └── login.html               # Login page
├── server/                      # Backend
│   ├── app.js                   # Express entry point (97 lines)
│   ├── .env                     # Environment variables (JWT_SECRET, PORT)
│   ├── db/
│   │   ├── database.js          # SQLite connection
│   │   ├── schema.js            # 12 table definitions
│   │   ├── seed.js              # Base seed data
│   │   └── univision.db         # Database file
│   ├── middleware/
│   │   └── auth.js              # JWT verification + role guards
│   ├── routes/
│   │   ├── auth.js              # Login/logout endpoints
│   │   ├── student.js           # Student API (8 GET routes)
│   │   ├── doctor.js            # Doctor API (20+ CRUD routes)
│   │   └── admin.js             # Admin API (1 POST route)
│   └── scripts/                 # Database seed scripts
│       ├── seed-doctor.js       # Seed doctor/admin users
│       ├── seed-students.js     # Seed 20 students across 4 grades
│       ├── seed-schedule.js     # Seed class schedules
│       └── seed-staff.js        # Seed course staff assignments
├── docs/
│   └── UniVision_Presentation.pptx
├── package.json                 # Node.js dependencies + scripts
└── README.md                    # Project documentation
```

14. Setup & Deployment Guide

Prerequisites

- Node.js v18+ and npm

Web Application Setup

```
# Clone the repository
git clone <repository-url>
cd rown

# Install dependencies
npm install

# Seed the database with sample data
npm run db:seed:all

# Start the server
npm start

# Server runs at http://localhost:3001
```

Demo Credentials

Role	ID	Password
Student	`30012151012341`	`123456`
Doctor	`29803050202394`	`123456`
Admin	`admin001`	`123456`

Available NPM Scripts

Command	Description
`npm start`	Start the server

Command	Description
`npm run dev`	Start with auto-reload (nodemon)
`npm run db:clean`	Wipe the database
`npm run db:seed:all`	Seed all data (doctor, students, schedule, staff)
`npm run db:seed:doctor`	Seed doctor/admin users only
`npm run db:seed:students`	Seed 20 students with grades/attendance
`npm run db:seed:schedule`	Seed class schedules
`npm run db:seed:staff`	Seed course staff assignments
`npm run db:seed:notifications`	Seed sample notifications
`npm run db:seed:materials`	Seed course materials (~55 resources)

15. AI Chatbot — UniBot

UniBot is a **rule-based, offline chatbot** that provides instant academic assistance without requiring any external API. It runs entirely on pattern matching and weighted keyword scoring, making it fast, reliable, and free of API quota limits.

Architecture

Pattern Matching Engine

Regex-based intent detection with weighted keyword scoring across 16 intents. Each message is scored against all intents and the highest-scoring match is selected.

Bilingual Support

Automatic language detection (English/Arabic). Responses are generated in the same language as the student's input.

Context-Aware Responses

Fetches real student data from the database (GPA, grades, attendance, schedule) for personalized, accurate responses.

Follow-Up Suggestions

Each response includes clickable suggestion buttons for guided conversation flow, reducing the need to type.

Supported Intents (16)

Intent	Example Queries	Response
`greeting`	"Hello", "مرحبا"	Welcome message with capabilities list
`gpa`	"What is my GPA?", "كم معدلي؟"	Current GPA, trend arrow, semester history
`grades`	"Show my grades", "عرض درجاتي"	All course grades with danger flags
`attendance`	"How is my attendance?"	Overall + per-course attendance
`schedule`	"What's my schedule today?"	Today's classes or full weekly schedule
`next_class`	"When is my next class?"	Finds the next upcoming class
`feedback`	"Any feedback from doctors?"	Doctor feedback messages

Intent	Example Queries	Response
`notifications`	"Any new notifications?"	Recent announcements
`profile`	"Show my profile"	Personal information summary
`danger`	"Any warnings or risks?"	At-risk courses, low GPA, attendance warnings
`best_course`	"What's my best course?"	Highest-scoring course
`worst_course`	"What's my worst course?"	Lowest-scoring course + improvement target
`tips`	"Give me study tips"	Personalized advice based on weak areas
`university`	"Tell me about the university"	EELU info: grading scale, probation, LMS
`thanks`	"Thank you", "شكرا"	Random appreciation response
`bye`	"Goodbye", "مع السلامة"	Farewell message

UI Features

- **Floating Action Button (FAB)** — Gold/purple gradient bubble in bottom-right with pulse animation
- **Slide-Up Chat Panel** — Header, scrollable message area, and input field
- **Typing Indicator** — Animated bouncing dots with 1.2–3 second simulated delay
- **Markdown Rendering** — Bot responses support bold, lists, and code formatting
- **Clear Chat** — Reset conversation with one click
- **Mobile Fullscreen** — Chat panel goes fullscreen on mobile (<480px) using `100dvh`

16. Progressive Web App (PWA)

UniVision is installable as a Progressive Web App on supported devices, providing an app-like experience.

Web App Manifest

Standalone display mode with no browser chrome. Theme color: `#d4a830` (gold), background: `#0a0e1a` (dark navy). Icons at 192×192 and 512×512.

Service Worker

Network-first strategy with cache fallback. Caches HTML, CSS, JS, logo, and Font Awesome. API requests are excluded (always network). Offline navigation falls back to cached home page.

Auto Registration

Service Worker registered via `theme.js` on every page. Manifest link and Apple meta tags injected dynamically.

Installation

Android/Chrome: Install prompt or browser menu. **iOS/Safari:** "Add to Home Screen" from the share menu (Chrome on iOS does not support PWA).

17. Notification System

A complete broadcast notification system connecting admins to students.

Categories

6 notification types: Day Off, Lecture Cancelled, Event, Holiday, Exam, General — each with a unique icon and color.

Date Grouping

Notifications organized by Today, Yesterday, This Week, and Older for easy scanning.

Read Tracking

Per-student read status tracked via `notification_reads` table. Click to mark read, or "Mark All Read" for one-click clearing.

Live Badge

Unread count badge on the notification bell icon in the navbar, updated in real-time on every page load.

Mobile Responsive

On mobile, the notification dropdown uses `position: fixed` to fill the viewport width, preventing overflow and ensuring readability on small screens.

18. Future Enhancements

Real-Time Messaging

WebSocket-based real-time chat between students and doctors for direct communication.

Assignment Tracking

Submission system with deadlines, file uploads, and grading workflow.

Analytics Dashboard

Admin-level analytics: department-wide GPA trends, attendance patterns, at-risk student identification.

Multi-Language Support

Full Arabic localization with RTL layout support for the entire portal.

Push Notifications

Firebase Cloud Messaging for real-time push alerts on mobile devices when new grades or notifications are posted.

AI-Powered Chatbot

Upgrade UniBot from rule-based to LLM-powered (e.g., Gemini/GPT) for natural language understanding and open-ended academic advising.

— End of Documentation —